IPCSDK for Android

1. Overview

Welcome to RayNeo IPCSDK for Android, an SDK designed specifically for third-party developers. It aims to provide a convenient interface to access relevant open data in glasses. With this SDK, you can easily access the open data of glasses, adding more innovation and interactivity to your application.

1. Overview

Welcome to use RayNeo IPCSDK for Android, an SDK designed for third-party developers to provide convenient interfaces for acquiring relevant open data from the glasses. With this SDK, you can easily access the open data of the glasses, adding more innovation and interactivity to your applications.

Adapted Glasses	Description	
Ring Information Acquisition	**X2**	Acquire the status information of the ring connected to the glasses, including connection status, IMU status, and quaternion data of the ring.
Ring Long-press Function Blocking	**X2**	The long-press of the ring globally triggers the glasses' Dock. The SDK allows blocking the ring long-press when the app is in the foreground, enabling developers to customize the long-press function of the ring for the current app.
Ring Touch Button Configuration	**X2**	Supports configuring whether the ring's Touchpad panel and the following buttons are integrated to achieve a personalized user experience.
Mobile GPS Data Streaming	**X2、X3**	Receive GPS data pushed from the mobile phone connected to the glasses.

2. IPC SDK File Download

Click to download

3. Quick Start

3.1 Introduce dependencies

3.1.1 rayneo Private Maven Access

If you can access the rayneo private Maven service, you can import the private Maven server in settings.gradle

Code block

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        // 配置雷鸟私有Maven服务器
        maven {
            url 'http://172.16.46.52:9998/repository/maven-public/'
            allowInsecureProtocol = true
        }
    }
}
```

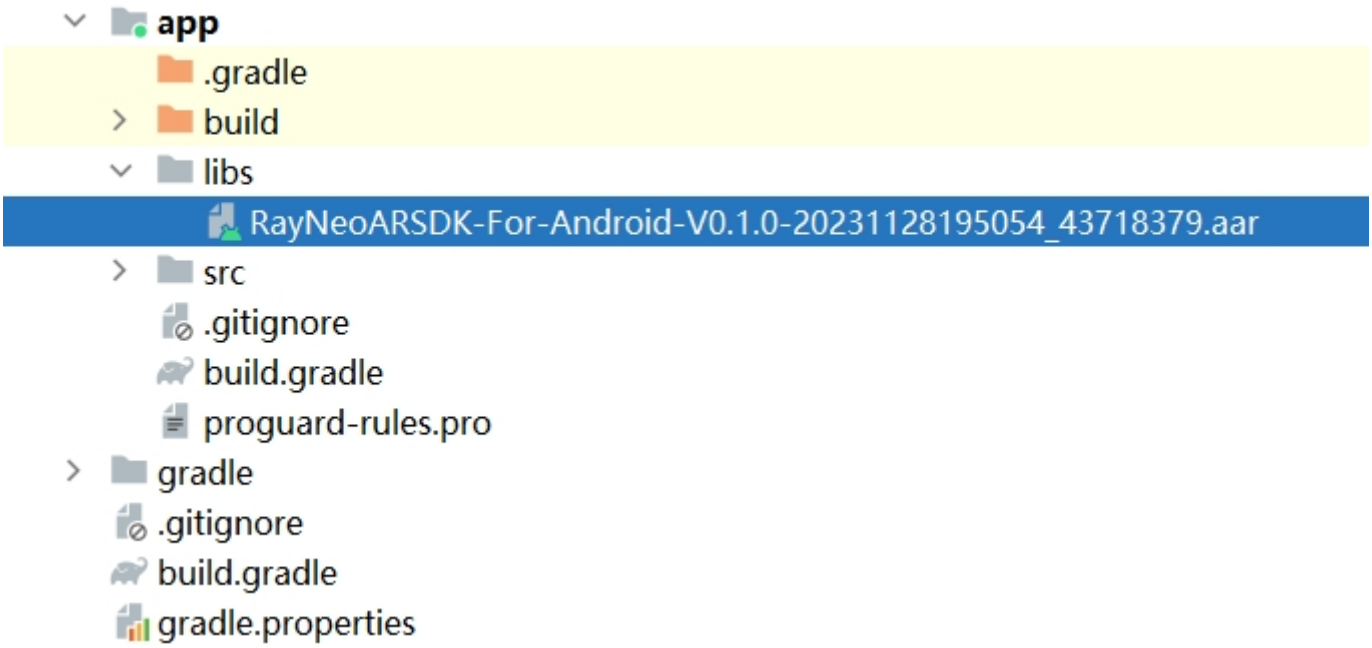
Introduce the following dependencies in the build.gradle of the main module of the project to complete the dependency addition:

Code block

```
implementation 'com.rayneo.arsdk:ipcsdk-for-android:0.1.0'
```

3.1.2 AAR file access

You can also just use the AAR file dependency access, first put the RayNeo IPCSDK for Android aar package into the libs directory of the main module, as shown in the figure below



3.2 Initialization

Create an instance of the `Launcher` class:

Code block

```
Launcher mLauncher = Launcher.getInstance(context);
```

Enable logging (SDK Log is disabled by default):

Code block

```
mLauncher.enableLog();//打开log
mLauncher.disableLog();//关闭Log，默认关闭
```

Add response listener:

Code block

```
mLauncher.addOnResponseListener(response); //绑定响应器
```

3.3 Close the connection

Remove response listener:

Code block

```
mLauncher.removeOnResponseListener(response);//解绑响应器
```

Disconnect: Perform the disconnect operation to release the connection with the glasses.

Code block

```
mLauncher.disConnect();
```

4. Feature use

4.1 Ring information

4.1.1 Register ring information

Code block

```
RingIPCHelper.registerRingInfo(this);//注册戒指信息，戒指连接、imu状态、四元数数据相关信息会返回
```

4.1.2 Cancel registration ring information

Code block

```
RingIPCHelper.unRegisterRingInfo(this);//取消注册戒指相关信息
```

4.1.3 Turn ring IMU data on/off

Code block

```
RingIPCHelper.setRingIMU(MainActivity.this, true);//开启IMU数据
RingIPCHelper.setRingIMU(MainActivity.this, false);//关闭IMU数据
```

4.1.4 Ring information return

Code block

```
private Launcher.OnResponseListener response = new Launcher.OnResponseListener() {
    @Override
```

```
    public void onResponse(Response response) {
        if (response == null || response.getData() == null)
            return;
        try {
            JSONObject jo = new JSONObject(response.getData());
            if (jo.has("datatype") &&
jo.getString("datatype").equals("ring_info")) {
                boolean isRingConnected = jo.getBoolean("ring_connected");
                int imuStatus = jo.getInt("ring_imu_status"); //-1: 戒指未连接,
0: 戒指未开启IMU 1: 戒指开启IMU
                if (isRingConnected) { //戒指连接
                    if (imuStatus != 1) {//IMU未开启
                        RingIPCHelper.setRingIMU(MainActivity.this, true);//开启
IMU
                    }
                } else {
                    Log.i(TAG, "戒指未连接 ");
                }
            } else if (jo.has("datatype") &&
jo.getString("datatype").equals("ring_quaternion")) {
                //获取四元数
                double w = jo.getDouble("w");
                double x = jo.getDouble("x");
                double y = jo.getDouble("y");
                double z = jo.getDouble("z");
            }
        } catch (JSONException e) {
            e.printStackTrace();
        }
    }
};
```

JSON Data Interpretation

Ring information (datatype: "ring_info")

Parameter name	Type	Description
ring_connected	Boolean	Whether the ring is connected, true: connected false: no connected
ring_imu_status	Integer	IMU status: -1 Ring not connected, 0 IMU not turned on, turned on

Quaternion information (datatype: "ring_quaternion")

Parameter name	Type	Description	Example values
w	Double	The value of quaternion w	0.7379355
x	Double	The x-value of a quaternion	-0.006100293
y	Double	The y value of a quaternion	0.003900187
z	Double	Z value of quaternion	0.6748324

4.2 Ring long press function shielding and recovery

By default, long pressing the ring will trigger the glasses Launcher. Third-party developers can block and restore the original long press function of the ring through the following interface

Code block

```
RingIPCHelper.setRingLongClick(this, false);//屏蔽戒指在当前应用显示时长按唤起Dock
RingIPCHelper.setRingLongClick(this, true);//恢复戒指在当前应用显示时长按唤起Dock
```

4.3 Ring Trackpad Separation

By default, the ring treats its TouchPad and its lower buttons as one entity and sends events to the user for processing in the form of MotionEvent. If some developers need to define more gesture categories and separate TouchPad and its lower button events, they can call the following API. After separation, some TouchPad touch events are sent in the form of MotionEvent, and its lower buttons are sent in the form of KeyEvent events (keycode 336).

Code block

```
RingIPCHelper.setRingSeparateButton(this, true);//button touchpad分离
RingIPCHelper.setRingSeparateButton(this, false);//button touchpad一体
```

4.4 Mobile GPS publishing streaming

Occasionally, there may be an unstable state in the acquisition of glasses gps information. Users can obtain the gps information of the mobile phone connected to the glasses by calling this api

4.4.1 Register GPS data publishing streaming

Code block

```
GPSIPCHelper.registerGPSInfo(this);//注册手机推流gps数据
```

4.4.2 Unregister GPS data publishing streaming

Code block

```
GPSIPCHelper.unRegisterGPSInfo(this);//取消手机推流gps数据
```

4.4.3 GPS information return

GPS publish streaming status information (datatype: " **gps_push** ")

Parameter name	Type	Description
resultCode	Integer	0 phone connected 1 phone not connected, phone not connected send (re-register publish streaming after user connects) 2. publish streaming timed out, please check phone connection status and try again
resultMessage	String	Mobile publishing streaming

GPS data information

Code block

```
privateLauncher.OnResponseListenerresponse=newLauncher.OnResponseListener() {
    @Override
    public voidonResponse(Response response) {
        if(response ==null|| response.getData() ==null)
            return;
        try{
        JSONObject jo = new JSONObject(response.getData());
        if(jo.has("mLatitude") && jo.has("mLongitude") && jo.has("mAltitude"))
{//GPS数据，之前与导航传输未封装datatype，兼容之前版本，未再次分装datatype
            String mProvider = jo.getString("mProvider");
            Long mTime = jo.getLong("mTime");
            Long mElapsedRealtimeNanos = jo.getLong("mElapsedRealtimeNanos");
            Double mLatitude = jo.getDouble("mLatitude");
            Double mLongitude = jo.getDouble("mLongitude");
            Double mAltitude = jo.getDouble("mAltitude");
            Double mSpeed = jo.getDouble("mSpeed");
```

```
16              Double mBearing = jo.getDouble("mBearing");
17              Double mHorizontalAccuracyMeters =
    jo.getDouble("mHorizontalAccuracyMeters");
18              Double mVerticalAccuracyMeters =
    jo.getDouble("mVerticalAccuracyMeters");
19              Double mSpeedAccuracyMetersPerSecond =
    jo.getDouble("mSpeedAccuracyMetersPerSecond");
20              Double mBearingAccuracyDegrees =
    jo.getDouble("mBearingAccuracyDegrees");
21          }catch(JSONException e) {
22              e.printStackTrace();
23          }
24      }
25  };
```

GPS data interpretation

Parameter name	Type	Description	Example values
mProvider	String	Location providers (GPS, network, etc.)	"gps"、"lbs"
mTime	Long	Position timestamp	1636580423000
mElapsedRealtimeNanos	Long		
mLatitude	Double	Latitude	37.7749
mLongitude	Double	Longitude	-122.4194
mAltitude	Double	Altitude (in meters)	0
mSpeed	Double	Speed (in meters per second)	5
mBearing	Double	Get direction angle (unit: degrees)	0
mHorizontalAccuracyMeters	Double	Acquire positioning accuracy in meters	10
mVerticalAccuracyMeters	Double		
mSpeedAccuracyMetersPerSecond	Double		
mBearingAccuracyDegrees	Double		

5. Update instructions

V0.1.0

Date of publication: 2023-11-28

- Completed the first version SDK encapsulation to achieve core functions such as ring information, quaternion data, long press frequency closure, touchpad separation, and mobile GPS publishing streaming acquisition